

操作日志系统实现方案

2026年03月14日 v1.0

操作日志系统实现方案

文档版本：v1.0

编写日期：2026 年 3 月 14 日

开发负责人：王舟

一、需求分析

1.1 功能需求

需求	说明	优先级
记录所有操作	增删改查全部记录	P0
永久保存	不删除，可追溯	P0
快速查询	按用户/时间/操作类型筛选	P0
详细信息	记录修改前后的值	P1
审计追踪	支持操作链还原	P1
导出功能	支持导出为 Excel/PDF	P2

1.2 记录范围

模块	操作类型	示例
用户管理	登录、登出、修改密码	用户登录系统
案件管理	创建、编辑、删除	创建新案件"张三诉李四"
文件管理	上传、解析、删除、关联	上传证据"合同.pdf"
节点管理	创建、编辑、删除、排序	创建节点"合同签订"
关联管理	绑定、解绑	节点关联证据
系统管理	配置修改、权限变更	修改用户角色

1.3 性能要求

指标	要求
写入性能	<50ms（异步写入，不阻塞主流程）
查询性能	<500ms（100 万条数据内）
存储周期	永久保存
数据量	预计 10 万条/年

二、数据库设计

2.1 表结构

```
-- 操作日志表
CREATE TABLE operation_logs (
```

```

-- 主键
id            TEXT PRIMARY KEY,

-- 基础信息
user_id       TEXT NOT NULL,           -- 操作人 ID
user_name     TEXT NOT NULL,           -- 操作人姓名（冗余，方便查询）
case_id       TEXT,                    -- 关联案件 ID（可选）

-- 操作信息
action        TEXT NOT NULL,           -- 操作类型：CREATE/UPDATE/DELETE/LOGIN/
module        TEXT NOT NULL,           -- 所属模块：case/file/node/user
target_type   TEXT NOT NULL,           -- 目标类型：evidence_file/event_node/ca
target_id     TEXT,                    -- 目标 ID
target_title  TEXT,                    -- 目标标题（冗余，方便展示）

-- 变更详情
old_value     JSON,                    -- 修改前的值（UPDATE/DELETE 时记录）
new_value     JSON,                    -- 修改后的值（CREATE/UPDATE 时记录）
changed_fields TEXT,                    -- 变更字段列表：逗号分隔，如 "title,time,d

-- 环境信息
ip_address    TEXT,                    -- 操作 IP
user_agent    TEXT,                    -- 浏览器/客户端信息
request_id    TEXT,                    -- 请求 ID（用于链路追踪）

-- 时间信息
created_at    DATETIME DEFAULT CURRENT_TIMESTAMP, -- 操作时间

-- 索引优化
INDEX idx_user_time (user_id, created_at),
INDEX idx_case_time (case_id, created_at),
INDEX idx_action_time (action, created_at),
INDEX idx_module_time (module, created_at),
INDEX idx_target (target_type, target_id)
);

```

2.2 索引策略

```

-- 单列索引
CREATE INDEX idx_logs_user_id ON operation_logs(user_id);
CREATE INDEX idx_logs_case_id ON operation_logs(case_id);
CREATE INDEX idx_logs_action ON operation_logs(action);
CREATE INDEX idx_logs_module ON operation_logs(module);
CREATE INDEX idx_logs_created_at ON operation_logs(created_at);

-- 复合索引（加速常用查询）

```

```

CREATE INDEX idx_logs_user_time ON operation_logs(user_id, created_at DESC);
CREATE INDEX idx_logs_case_time ON operation_logs(case_id, created_at DESC);
CREATE INDEX idx_logs_module_action ON operation_logs(module, action);

-- 全文索引（加速内容搜索）
CREATE VIRTUAL TABLE logs_search USING fts5(
    user_name,
    target_title,
    action,
    module,
    changed_fields,
    content='operation_logs',
    content_rowid='rowid'
);

-- 触发器：自动同步搜索索引
CREATE TRIGGER logs_ai AFTER INSERT ON operation_logs BEGIN
    INSERT INTO logs_search(rowid, user_name, target_title, action, module, changed_fields)
    VALUES (NEW.rowid, NEW.user_name, NEW.target_title, NEW.action, NEW.module, NEW.changed_fields);
END;

```

2.3 分区策略（可选，数据量大时使用）

```

-- 按年分区（SQLite 不原生支持，可用表模拟）
CREATE TABLE operation_logs_2026 (
    -- 同主表结构
    CHECK (created_at >= '2026-01-01' AND created_at < '2027-01-01')
);

CREATE TABLE operation_logs_2027 (
    -- 同主表结构
    CHECK (created_at >= '2027-01-01' AND created_at < '2028-01-01')
);

```

三、Rust 后端实现

3.1 数据模型

```
// src/models/log.rs
use chrono::{DateTime, Utc};
use serde::{Deserialize, Serialize};
use sqlx::FromRow;
use uuid::Uuid;

/// 操作类型枚举
#[derive(Debug, Clone, Serialize, Deserialize, PartialEq)]
#[serde(rename_all = "SCREAMING_SNAKE_CASE")]
pub enum ActionType {
    Create,
    Update,
    Delete,
    Login,
    Logout,
    Upload,
    Download,
    Parse,
    Link,
    Unlink,
    Export,
}

/// 所属模块
#[derive(Debug, Clone, Serialize, Deserialize, PartialEq)]
#[serde(rename_all = "lowercase")]
pub enum ModuleType {
    Case,
    File,
    Node,
    User,
    System,
}

/// 操作日志结构
#[derive(Debug, Clone, FromRow, Serialize, Deserialize)]
pub struct OperationLog {
    pub id: String,
    pub user_id: String,
    pub user_name: String,
```

```

    pub case_id: Option<String>,
    pub action: String,
    pub module: String,
    pub target_type: String,
    pub target_id: Option<String>,
    pub target_title: Option<String>,
    pub old_value: Option<serde_json::Value>,
    pub new_value: Option<serde_json::Value>,
    pub changed_fields: Option<String>,
    pub ip_address: Option<String>,
    pub user_agent: Option<String>,
    pub request_id: Option<String>,
    pub created_at: DateTime<Utc>,
}

```

```

impl OperationLog {
    /// 创建新日志
    pub fn new(
        user_id: String,
        user_name: String,
        action: ActionType,
        module: ModuleType,
        target_type: String,
    ) -> Self {
        Self {
            id: Uuid::new_v4().to_string(),
            user_id,
            user_name,
            case_id: None,
            action: format!("{:?}", action),
            module: format!("{:?}", module),
            target_type,
            target_id: None,
            target_title: None,
            old_value: None,
            new_value: None,
            changed_fields: None,
            ip_address: None,
            user_agent: None,
            request_id: None,
            created_at: Utc::now(),
        }
    }
}

```

```

/// 设置变更详情
pub fn with_changes(

```

```

        mut self,
        old_value: Option<serde_json::Value>,
        new_value: Option<serde_json::Value>,
    ) -> Self {
        self.changed_fields = Self::extract_changed_fields(&old_value, &new_value);
        self.old_value = old_value;
        self.new_value = new_value;
        self
    }

    /// 提取变更字段列表
    fn extract_changed_fields(
        old_value: &Option<serde_json::Value>,
        new_value: &Option<serde_json::Value>,
    ) -> Option<String> {
        match (old_value, new_value) {
            (Some(Value::Object(old)), Some(Value::Object(new))) => {
                let changed: Vec<String> = old
                    .keys()
                    .filter(|k| old.get(*k) != new.get(*k))
                    .cloned()
                    .collect();
                if changed.is_empty() {
                    None
                } else {
                    Some(changed.join(","))
                }
            }
            _ => None,
        }
    }
}

```

3.2 日志服务

```

// src/services/log_service.rs
use axum::extract::State;
use sqlx::{SqlitePool, FromRow};
use crate::models::log::OperationLog;
use crate::AppState;

pub struct LogService {
    pool: SqlitePool,
}

```

```

impl LogService {
    pub fn new(pool: SqlitePool) -> Self {
        Self { pool }
    }

    /// 记录日志 (同步写入, 简单场景)
    pub async fn log(&self, log: OperationLog) -> anyhow::Result<()> {
        sqlx::query(
            r#"
            INSERT INTO operation_logs (
                id, user_id, user_name, case_id, action, module,
                target_type, target_id, target_title, old_value, new_value,
                changed_fields, ip_address, user_agent, request_id
            ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
            "#,
        )
        .bind(&log.id)
        .bind(&log.user_id)
        .bind(&log.user_name)
        .bind(&log.case_id)
        .bind(&log.action)
        .bind(&log.module)
        .bind(&log.target_type)
        .bind(&log.target_id)
        .bind(&log.target_title)
        .bind(&log.old_value)
        .bind(&log.new_value)
        .bind(&log.changed_fields)
        .bind(&log.ip_address)
        .bind(&log.user_agent)
        .bind(&log.request_id)
        .execute(&self.pool)
        .await?;

        Ok(())
    }

    /// 查询日志列表
    pub async fn query_logs(
        &self,
        user_id: Option<&str>,
        case_id: Option<&str>,
        module: Option<&str>,
        action: Option<&str>,
        start_time: Option<DateTime<Utc>>,
        end_time: Option<DateTime<Utc>>,
    ) -> anyhow::Result<Vec<OperationLog>> {
        sqlx::query(
            r#"
            SELECT * FROM operation_logs
            WHERE
                user_id = ?
                AND case_id = ?
                AND module = ?
                AND action = ?
                AND start_time <= ?
                AND end_time >= ?
            "#,
        )
        .bind(user_id)
        .bind(case_id)
        .bind(module)
        .bind(action)
        .bind(start_time)
        .bind(end_time)
        .execute(&self.pool)
        .await?;

        Ok(sqlx::query_map(
            r#"
            SELECT * FROM operation_logs
            WHERE
                user_id = ?
                AND case_id = ?
                AND module = ?
                AND action = ?
                AND start_time <= ?
                AND end_time >= ?
            "#,
            [
                user_id,
                case_id,
                module,
                action,
                start_time,
                end_time,
            ],
        ).execute(&self.pool).await?.rows)
    }
}

```



```

        keyword: Option<&str>,
        page: i64,
        page_size: i64,
    ) -> anyhow::Result<(Vec<OperationLog>, i64)> {
        // 构建动态查询
        let mut where_clauses = Vec::new();
        let mut binds: Vec<Box<dyn sqlx::Encode<sqlx::Sqlite> + Send>> = Vec::new();

        if let Some(uid) = user_id {
            where_clauses.push("user_id = ?");
            binds.push(Box::new(uid));
        }

        if let Some(cid) = case_id {
            where_clauses.push("case_id = ?");
            binds.push(Box::new(cid));
        }

        if let Some(m) = module {
            where_clauses.push("module = ?");
            binds.push(Box::new(m));
        }

        if let Some(a) = action {
            where_clauses.push("action = ?");
            binds.push(Box::new(a));
        }

        if let Some(start) = start_time {
            where_clauses.push("created_at >= ?");
            binds.push(Box::new(start));
        }

        if let Some(end) = end_time {
            where_clauses.push("created_at <= ?");
            binds.push(Box::new(end));
        }

        if let Some(kw) = keyword {
            // 使用 FTS5 搜索
            where_clauses.push("target_title LIKE ?");
            binds.push(Box::new(format!("{}%", kw)));
        }

        let where_clause = if where_clauses.is_empty() {
            String::new()
        } else {

```

```

        format!("WHERE {}", where_clauses.join(" AND "))
    };

    // 查询总数
    let count_query = format!(
        "SELECT COUNT(*) FROM operation_logs {}",
        where_clause
    );
    let total: (i64,) = sqlx::query_as(&count_query)
        .fetch_one(&self.pool)
        .await?;

    // 查询数据
    let data_query = format!(
        r#"
        SELECT id, user_id, user_name, case_id, action, module,
               target_type, target_id, target_title, old_value, new_value,
               changed_fields, ip_address, user_agent, request_id, created_at
        FROM operation_logs
        {}
        ORDER BY created_at DESC
        LIMIT ? OFFSET ?
        "#,
        where_clause
    );

    let logs: Vec<OperationLog> = sqlx::query_as(&data_query)
        .fetch_all(&self.pool)
        .await?;

    Ok((logs, total.0))
}

/// 获取操作历史（针对特定目标）
pub async fn get_target_history(
    &self,
    target_type: &str,
    target_id: &str,
) -> anyhow::Result<Vec<OperationLog>> {
    let logs = sqlx::query_as:::<_, OperationLog>(
        r#"
        SELECT * FROM operation_logs
        WHERE target_type = ? AND target_id = ?
        ORDER BY created_at DESC
        LIMIT 100
        "#,
    )
}

```

```

        .bind(target_type)
        .bind(target_id)
        .fetch_all(&self.pool)
        .await?;

    Ok(logs)
}

/// 统计操作次数
pub async fn count_actions(
    &self,
    user_id: &str,
    start_time: DateTime<Utc>,
    end_time: DateTime<Utc>,
) -> anyhow::Result<Vec<(String, i64)>> {
    let stats = sqlx::query_as::<_, (String, i64)>(
        r#"
        SELECT action, COUNT(*) as count
        FROM operation_logs
        WHERE user_id = ? AND created_at BETWEEN ? AND ?
        GROUP BY action
        ORDER BY count DESC
        "#,
    )
    .bind(user_id)
    .bind(start_time)
    .bind(end_time)
    .fetch_all(&self.pool)
    .await?;

    Ok(stats)
}
}

```

3.3 日志宏（简化调用）

```

// src/macros.rs

/// 记录操作日志宏
#[macro_export]
macro_rules! log_operation {
    (
        $ctx:expr,
        action: $action:expr,
        module: $module:expr,
    )
}

```

```

        target_type: $target_type:expr,
        target_id: $target_id:expr,
        target_title: $target_title:expr,
        old: $old:expr,
        new: $new:expr
    ) => {
        let log = $crate::models::log::OperationLog::new(
            $ctx.user_id.clone(),
            $ctx.user_name.clone(),
            $action,
            $module,
            $target_type.to_string(),
        )
        .with_changes($old, $new);

        // 异步写入, 不阻塞主流程
        let log_service = $ctx.log_service.clone();
        tokio::spawn(async move {
            if let Err(e) = log_service.log(log).await {
                tracing::error!("Failed to write log: {}", e);
            }
        });
    };

    // 简化版 (无变更详情)
    (
        $ctx:expr,
        action: $action:expr,
        module: $module:expr,
        target_type: $target_type:expr,
        target_id: $target_id:expr
    ) => {
        log_operation!(
            $ctx,
            action: $action,
            module: $module,
            target_type: $target_type,
            target_id: $target_id,
            target_title: None,
            old: None,
            new: None
        );
    };
}

```

3.4 使用示例

```
// src/routers/nodes.rs
use axum::{extract::{State, Json}};
use crate::AppState;
use crate::models::log::{ActionType, ModuleType};
use crate::log_operation;

/// 创建事件节点
pub async fn create_node(
    State(state): State<Arc<AppState>>,
    ctx: AppContext,
    Json(input): Json<CreateNodeInput>,
) -> Result<Json<EventNode>> {
    // 创建节点
    let node = state.db.create_node(&input).await?;

    // 记录日志
    log_operation!(
        ctx,
        action: ActionType::Create,
        module: ModuleType::Node,
        target_type: "event_node",
        target_id: node.id.clone(),
        target_title: Some(node.title.clone()),
        old: None,
        new: Some(json!(&node))
    );

    Ok(Json(node))
}

/// 更新事件节点
pub async fn update_node(
    State(state): State<Arc<AppState>>,
    ctx: AppContext,
    Path(node_id): Path<String>,
    Json(input): Json<UpdateNodeInput>,
) -> Result<Json<EventNode>> {
    // 获取旧值
    let old_node = state.db.get_node(&node_id).await?;

    // 更新节点
    let new_node = state.db.update_node(&node_id, &input).await?;

    // 记录日志
```

```

        log_operation!(
            ctx,
            action: ActionType::Update,
            module: ModuleType::Node,
            target_type: "event_node",
            target_id: node_id.clone(),
            target_title: Some(new_node.title.clone()),
            old: Some(json!(&old_node)),
            new: Some(json!(&new_node))
        );

        Ok(Json(new_node))
    }

    /// 删除事件节点
    pub async fn delete_node(
        State(state): State<Arc<AppState>>,
        ctx: AppContext,
        Path(node_id): Path<String>,
    ) -> Result<()> {
        // 获取旧值
        let old_node = state.db.get_node(&node_id).await?;

        // 删除节点
        state.db.delete_node(&node_id).await?;

        // 记录日志
        log_operation!(
            ctx,
            action: ActionType::Delete,
            module: ModuleType::Node,
            target_type: "event_node",
            target_id: node_id.clone(),
            target_title: Some(old_node.title.clone()),
            old: Some(json!(&old_node)),
            new: None
        );

        Ok(())
    }
}

```

四、API 接口设计

4.1 查询日志列表

```
GET /api/v1/logs
```

请求参数：

参数	类型	必填	说明
user_id	string	否	按用户筛选
case_id	string	否	按案件筛选
module	string	否	按模块筛选：case/file/node
action	string	否	按操作类型筛选：CREATE/UPDATE/DELETE
start_time	datetime	否	开始时间
end_time	datetime	否	结束时间
keyword	string	否	关键词搜索
page	int	否	页码，默认 1
page_size	int	否	每页数量，默认 20

响应示例：

```
{
  "code": 200,
  "data": {
    "logs": [
      {
        "id": "uuid-1",
        "user_id": "user-1",
        "user_name": "张三",
```

```

        "case_id": "case-1",
        "action": "CREATE",
        "module": "node",
        "target_type": "event_node",
        "target_id": "node-1",
        "target_title": "合同签订",
        "old_value": null,
        "new_value": {
            "id": "node-1",
            "title": "合同签订",
            "event_time": "2024-01-01"
        },
        "changed_fields": null,
        "ip_address": "192.168.1.100",
        "created_at": "2026-03-14T10:30:00Z"
    },
    {
        "id": "uuid-2",
        "user_id": "user-1",
        "user_name": "张三",
        "case_id": "case-1",
        "action": "UPDATE",
        "module": "node",
        "target_type": "event_node",
        "target_id": "node-1",
        "target_title": "合同签订",
        "old_value": {
            "event_time": "2024-01-01"
        },
        "new_value": {
            "event_time": "2024-01-02"
        },
        "changed_fields": "event_time",
        "ip_address": "192.168.1.100",
        "created_at": "2026-03-14T11:00:00Z"
    }
],
"total": 156,
"page": 1,
"page_size": 20
}

```


4.2 获取操作历史

```
GET /api/v1/logs/:target_type/:target_id/history
```

响应示例:

```
{
  "code": 200,
  "data": {
    "target_type": "event_node",
    "target_id": "node-1",
    "history": [
      {
        "action": "CREATE",
        "user_name": "张三",
        "created_at": "2026-03-14T10:30:00Z",
        "changes": null
      },
      {
        "action": "UPDATE",
        "user_name": "张三",
        "created_at": "2026-03-14T11:00:00Z",
        "changes": {
          "event_time": {
            "old": "2024-01-01",
            "new": "2024-01-02"
          }
        }
      },
      {
        "action": "UPDATE",
        "user_name": "李四",
        "created_at": "2026-03-14T14:00:00Z",
        "changes": {
          "description": {
            "old": null,
            "new": "补充说明"
          }
        }
      }
    ]
  }
}
```

4.3 导出日志

```
GET /api/v1/logs/export?format=excel&user_id=xxx&start_time=xxx&end_time=xxx
```

五、Dioxus 前端实现

5.1 日志列表页面

```
// frontend/src/pages/logs.rs
use dioxus::prelude::*;
use crate::components::{Table, Pagination, SearchBar};

#[component]
pub fn LogsPage() -> Element {
    let mut filters = use_signal(|| LogFilters::default());
    let logs = use_resource(move || fetch_logs(filters()));

    rsx! {
        div { class: "logs-page",
            h1 { "操作日志" }

            // 搜索栏
            SearchBar {
                filters: filters(),
                on_search: move |f| filters.set(f),
            }

            // 日志列表
            match &*logs.value() {
                Some(Ok(data)) => rsx! {
                    Table {
                        columns: vec![
                            Column { header: "操作时间", field: "created_at" },
                            Column { header: "操作人", field: "user_name" },
                            Column { header: "操作类型", field: "action" },
                            Column { header: "模块", field: "module" },
                            Column { header: "目标", field: "target_title" },
                            Column { header: "变更字段", field: "changed_fields" },
                            Column { header: "详情", field: "actions" },
                        ],
                    }
                }
            }
        }
    }
}
```

```

        data: data.logs.clone(),
    }

    Pagination {
        current_page: data.page,
        total_pages: (data.total / data.page_size) + 1,
        on_page_change: move |page| {
            filters.write().page = page;
        },
    }
},
Some(Err(e)) => rsx! {
    div { class: "error", "加载失败: {e}" }
},
None => rsx! {
    div { class: "loading", "加载中..." }
},
}
}
}
}
}

```

5.2 操作历史侧边栏

```

// frontend/src/components/OperationHistory.rs
#[component]
pub fn OperationHistory(target_type: String, target_id: String) -> Element {
    let history = use_resource(move || fetch_history(&target_type, &target_id));

    rsx! {
        div { class: "operation-history",
            h4 { "操作历史" }

            match &*history.value() {
                Some(Ok(items)) => rsx! {
                    ul { class: "timeline",
                        for item in items.iter() {
                            li { class: "timeline-item",
                                span { class: "time", "{item.created_at}" }
                                span { class: "user", "{item.user_name}" }
                                span { class: "action", "{item.action}" }

                                if let Some(changes) = &item.changes {
                                    div { class: "changes",
                                        for (field, change) in changes.iter() {

```

```
      div {  
        span { "{field}: " }  
        span { class: "old", "{change.old}" }  
        span { " → " }  
        span { class: "new", "{change.new}" }  
      }  
    }  
  }  
},  
_ => rsx! { div { "暂无操作历史" } },  
}  
  
}
```

六、性能优化

6.1 异步写入

```
// 使用消息队列异步写入日志
use tokio::sync::mpsc;

pub struct LogWriter {
    sender: mpsc::Sender<OperationLog>,
}

impl LogWriter {
    pub fn new(pool: SqlitePool) -> Self {
        let (sender, mut receiver) = mpsc::channel(1000);

        // 后台任务批量写入
        tokio::spawn(async move {
            let mut buffer = Vec::with_capacity(100);

            while let Some(log) = receiver.recv().await {
                buffer.push(log);

                // 批量写入 (100 条或 1 秒)
```

```

        if buffer.len() >= 100 {
            let _ = bulk_insert_logs(&pool, &buffer).await;
            buffer.clear();
        }
    }
});

Self { sender }
}

pub async fn write(&self, log: OperationLog) {
    let _ = self.sender.send(log).await;
}
}

```

6.2 定期归档

```

// 定期将旧数据归档到冷存储
pub async fn archive_old_logs(pool: &SqlitePool, days: i32) {
    let cutoff = Utc::now() - ChronoDuration::days(days as i64);

    // 导出到 CSV
    let csv_path = format!("./archive/logs_{}.csv", Utc::now().format("%Y%m%d"));
    export_to_csv(pool, cutoff, &csv_path).await.unwrap();

    // 删除旧数据（可选，根据合规要求决定）
    // sqlx::query("DELETE FROM operation_logs WHERE created_at < ?")
    //     .bind(cutoff)
    //     .execute(pool)
    //     .await.unwrap();
}

```

七、测试用例

```

#[cfg(test)]
mod tests {
    use super::*;

    #[tokio::test]
    async fn test_log_creation() {

```

```
let pool = setup_test_db().await;
let service = LogService::new(pool);

let log = OperationLog::new(
    "user-1".to_string(),
    "张三".to_string(),
    ActionType::Create,
    ModuleType::Node,
    "event_node".to_string(),
);

service.log(log).await.unwrap();

let (logs, total) = service.query_logs(
    Some("user-1"),
    None,
    None,
    None,
    None,
    None,
    None,
    1,
    20,
)
.await
.unwrap();

assert_eq!(total, 1);
assert_eq!(logs[0].user_name, "张三");
}
}
```

八、总结

模块	技术选型	说明
数据库	SQLite + FTS5	简单够用，支持全文搜索
写入方式	异步写入	不阻塞主流程

模块	技术选型	说明
查询性能	复合索引	<500ms 响应
存储周期	永久保存	定期归档到 CSV
前端展示	Dioxus	时间轴形式展示历史

文档版本 : v1.0

最后更新: 2026 年 3 月 14 日

开发负责人: 王舟

邮箱 : main@mails.wedevs.org

开发负责人: 王舟 | 电话: 15378391447 | 邮箱: main@mails.wedevs.org